

Fundamentos

Trabajar con agentes: prompt, contexto y sistema

Para quién

Cualquiera que trabaje con agentes, use o no el framework. Es el documento de entrada: los fundamentos que sostienen todo lo demás, sin depender de ninguna herramienta concreta.

Qué resuelve

La confusión entre tres oficios distintos —redactar el prompt, decidir el contexto, diseñar el sistema— que hace que la gente intente arreglar con redacción un problema de arquitectura. Y da las tres técnicas concretas contra la respuesta inventada.

1 · Los tres niveles

Hay tres oficios distintos que se confunden todo el tiempo. Confundirlos hace que la gente reformule veinte veces algo que fallaba por falta de información.

Nivel	Pregunta	Si falla
Prompt	¿Cómo lo escribo?	Reformulas
Contexto	¿Qué necesita saber ahora?	Revisas de dónde salió la información
Sistema	¿Quién decide y con qué evidencia?	Revisas quién tenía autoridad para aprobarlo

Los tres hacen falta. Un prompt impecable con el contexto equivocado produce una respuesta impecable y equivocada — y esa no la revisa nadie, porque se ve bien.

2 · Cómo funciona por dentro

Lo mínimo que hay que entender para que el resto tenga sentido.

Las palabras tienen ubicación

Un modelo asigna a cada palabra una posición en un espacio de muchas dimensiones, donde la cercanía representa cercanía de significado. Rey está cerca de reina; hospital, de médico. Tanto que la aritmética funciona: rey – hombre + mujer cae donde está reina.

Por qué importa: cada palabra que escribes mueve la atención hacia una región de ese espacio. No estás dando instrucciones a un programa; estás eligiendo desde qué zona del conocimiento va a responder.

De ahí una recomendación poco obvia: escribe en tu lengua nativa. En un idioma que no dominas se pierden sutilezas y matices culturales que sí sabes usar en el propio — y esas sutilezas son exactamente lo que mueve la atención.

El rol no es decoración

Asignar un rol es el mecanismo más barato para mover la atención a la región correcta. No es maquillaje de tono: cambia de qué zona del espacio sale la respuesta.

La práctica que casi nadie hace

Antes de asignar un rol, pregúntale al modelo qué entiende él por ese rol. Muchas veces crees que necesitas un «community manager» y resulta que lo que hace falta es un guionista. Lo descubres preguntando, no suponiendo.

Delegar, no ordenar

El encuadre más útil: trata al modelo como alguien competente que acaba de entrar y no conoce tu contexto. Cuando delegamos mal a una persona, es fácil culparla y no revisar cómo delegamos. Con un modelo pasa igual, con la diferencia de que nunca te va a decir «no entendí».

3 · Estructura de una instrucción

Cuatro piezas. No es una regla de oro, sino una lista de preguntas para saber qué le falta a lo que escribiste.

Pieza	Qué responde	Error típico
Rol	¿Desde qué conocimiento respondes?	Elegirlo por intuición sin preguntar qué entiende el modelo
Enfoque	¿Qué tiene que hacer?	Decorar la instrucción. No hace falta pedir el favor
Contexto	¿Qué información necesita?	Suponer que ya lo sabe
Límites	¿Qué forma tiene el resultado?	Decir «corto» sin decir cuánto

Sobre los límites: «corto» no significa nada. En un libro, ochenta páginas es corto. Cuantifica —caracteres, oraciones, viñetas— y añade adjetivos que orienten: accionable, conciso, pragmático.

Destilar el prompt

Cuando el problema es nuevo, iterar en conversación está bien. Lo que casi nadie hace es el paso final. Una vez llegaste al resultado que querías:

```
¿Cuál hubiera sido el prompt que debí darte al inicio  
para llegar a este resultado?
```

El modelo devuelve la instrucción destilada. Eso convierte una conversación de diez turnos en un prompt reutilizable — y es la forma más barata de fabricar la primera versión de una skill.

4 · Las cuatro decisiones de contexto

Decisión	Gobierna	Pregunta de auditoría
Relevancia	Qué entra	Si quito esta línea, ¿cambia alguna respuesta?
Recencia	Qué sale	Lo viejo, ¿se muere o solo se acumula?
Recuperación	Cuándo entra	¿Siempre, o solo cuando lo pidan?
Ranking	Dónde se coloca	Lo importante, ¿está donde más se ve?

Dos consecuencias que sorprenden:

- Más no es mejor. En un benchmark de llamadas a funciones, un modelo falló con 46 herramientas y funcionó con 19. Quitar capacidades lo mejoró.
- Las definiciones de herramientas ocupan contexto aunque no se usen. Cuarenta declaradas y seis en uso: pagas las cuarenta en cada mensaje.

Y el orden importa: la misma información rinde 70–75% de exactitud al inicio o al final de la ventana, y 55–60% en la mitad. De ahí las tres zonas — ancla (estable, se cachea) → volumen (lo pesado, al medio) → filo (la tarea, siempre al final). Nada después del filo.

5 · Dónde poner cada dato

Tres preguntas en cadena resuelven casi todas las decisiones de ubicación. Cabe en una tarjeta.

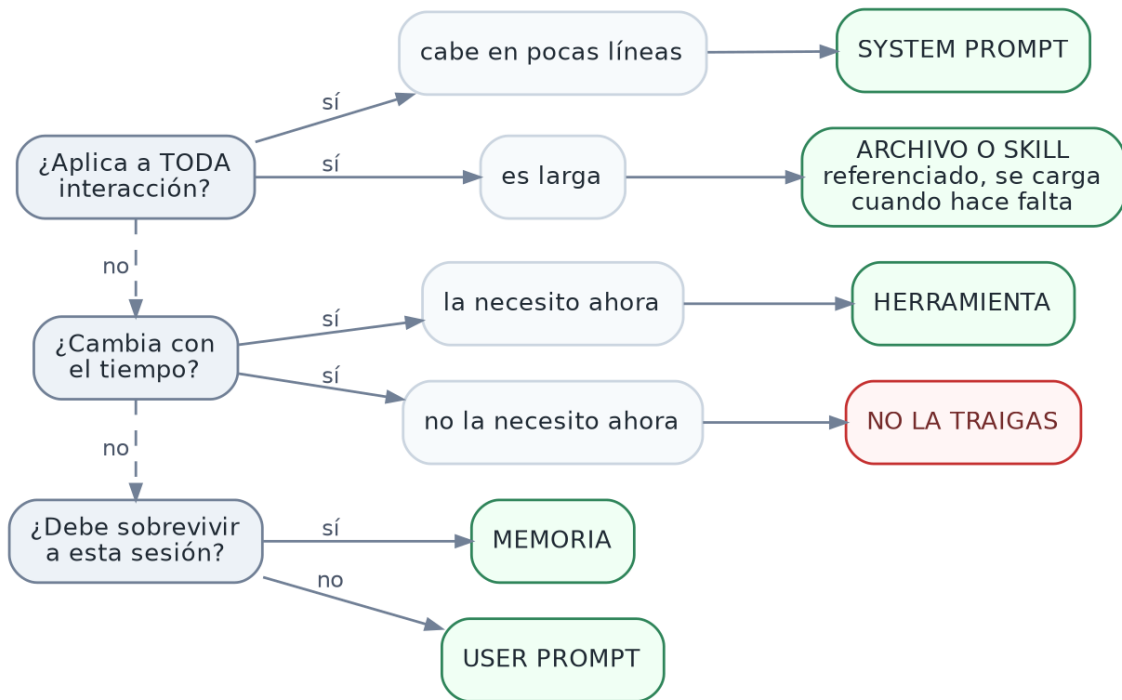


Figura 1 — Árbol de decisión. Las tres preguntas son, en orden, relevancia (¿aplica siempre?), recencia (¿cambia?) y recuperación (¿la necesito ahora?).

Dos ramas merecen atención porque son las que más se hacen mal:

- «Sí, pero es larga» → archivo referenciado, no system prompt. La tentación es pegar el manual entero arriba. Eso lo mete en cada llamada aunque casi nunca haga falta, y lo empuja al medio de la ventana, donde peor rinde.
- «Cambia y no la necesito ahora» → no la traigas. No es «tráela por si acaso». Un dato que cambia y no se usa hoy solo puede hacer una cosa: estar desactualizado cuando alguien lo lea.

6 · Auditar qué está sobrando

El árbol dice dónde poner las cosas. Esta matriz dice qué quitar de lo que ya pusiste. Corre la tarea varias veces, anota qué información trajiste y responde dos preguntas: ¿cambió la respuesta? y ¿ya lo tenía en contexto?

	NO lo tenía en contexto	SÍ lo tenía en contexto
CAMBIÓ la respuesta	FALTABA señal real → súbela al system prompt o a una herramienta	MAL UBICADA ahí no la usaba → acorta o mueve, no la dupliques
NO cambió la respuesta	RUIDO no aporta → no la traigas más	DUPLICADO ya estaba y funcionaba → deja de repetirla

Figura 2 — Cuatro diagnósticos, cuatro acciones distintas. La misma tabla sirve para decidir qué subir y qué borrar.

El cuadrante que más enseña

«Sí cambió y ya lo tenía» no significa que faltara: significa que donde estaba no la estaba usando — enterrada en un bloque largo o en la mitad de la ventana. Meterla dos veces no es la solución. Acortarla o reubicarla, sí.

Córrela en sesiones limpias, una por variante. En la misma sesión, lo que trajiste antes contamina lo siguiente y la tabla miente.

7 · Encadenar en vez de agrandar

La reacción natural ante una tarea compleja es escribir un prompt más largo y meterlo todo en una conversación. Pero la ventana crece, y con ella crecen los errores, las invenciones y las instrucciones olvidadas. Un prompt gigante es la peor forma de resolver un problema grande.

Nadie construye un coche de una sola vez. Hay una estación que monta el chasis, otra el motor, otra lo eléctrico. Cada una sabe exactamente qué tiene que resolver y recibe el trabajo de la anterior. Con los prompts funciona igual: cada paso es un prompt en un chat nuevo.

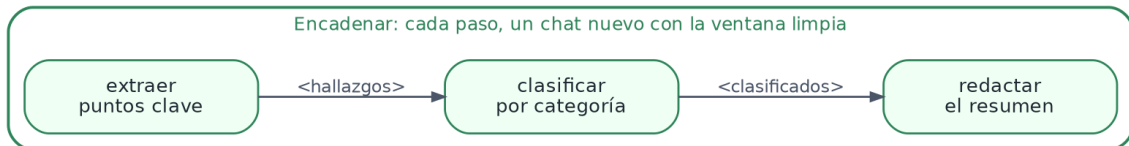


Figura 3 — Cada estación arranca con la ventana limpia y una sola tarea. Lo que arrastra no es el documento entero, sino el resultado destilado.

Pasar el resultado con etiquetas

A partir de los siguientes hallazgos, escribe un resumen de 100 palabras enfocado en las soluciones más viables.

... la salida del paso anterior ...

Sin las etiquetas, el modelo no distingue tu instrucción del material, y a veces responde sobre el material en lugar de hacer lo que le pediste.

La ventaja que no es obvia

Cada paso es un prompt que puedes iterar por separado hasta dejarlo bien. Una vez afinado, la cadena entera produce la misma calidad cada vez. Con un prompt gigante no puedes hacer eso: si el resultado sale mal, no sabes qué parte falló.

8 · Fabricar el procedimiento

Cómo conseguir el paso a paso de una tarea sin inventártelo. Tres movimientos.

1. ¿Qué harías paso a paso para resolver este tipo de problemas? Dame tres alternativas.
2. [eliges la que mejor resuelve tu caso]
3. Usando esta plantilla, resuelve el problema que te daré.
...
...

Dos efectos, y el segundo importa más de lo que parece:

- Acota el razonamiento. Al darle el paso a paso, limitas los caminos que puede tomar. Más rápido y más consistente entre ejecuciones.

- Resuelve el problema de «qué palabras uso». La plantilla viene formulada con las palabras del propio modelo para razonar sobre ese tipo de problema — mejores que las que tú adivinarías desde fuera.

Combinado con la destilación del prompt, es la forma más barata de fabricar la primera versión de una skill: el modelo te da el procedimiento y la instrucción que lo invoca. Tú decides cuál de las alternativas es correcta, que es la parte que no se delega.

9 · Orden, caché y costo

Dos optimizaciones que tiran en direcciones opuestas si no las ves juntas:

- La caché exige que el contenido estable vaya primero: el proveedor reutiliza el cómputo del prefijo idéntico entre llamadas. Hay medición de reducciones de costo de API entre 41% y 80%, y de mejora del time-to-first-token entre 13% y 31%.
- La atención exige que lo importante vaya al principio o al final: 70–75% de exactitud en los extremos, 55–60% en la mitad.

Puestas juntas mal, el prefijo estable empuja la tarea real hacia la mitad —la peor posición— y se gana en costo lo que se pierde en exactitud.

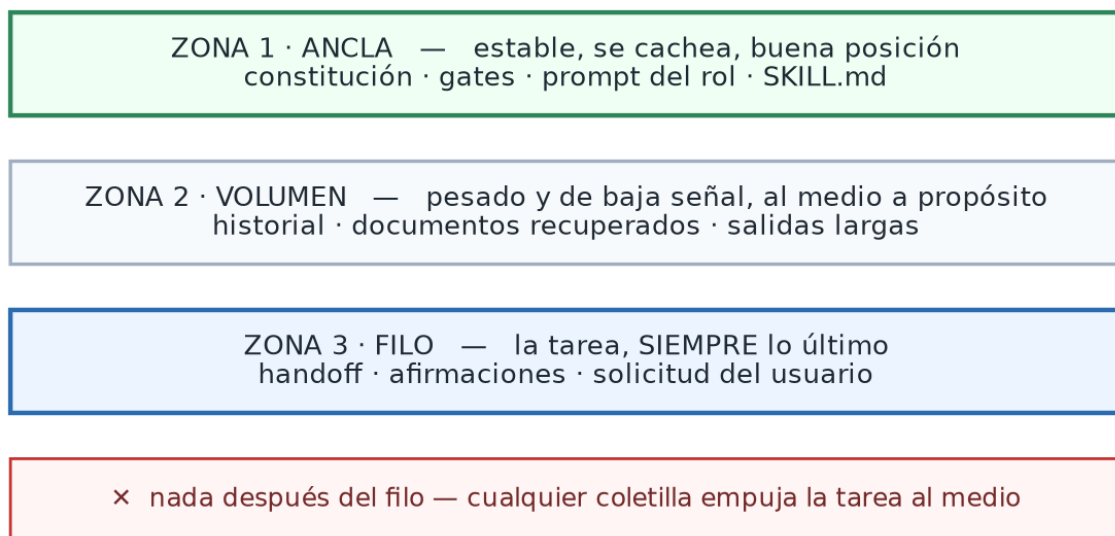


Figura 4 — Tres zonas lo resuelven. El ancla ocupa el inicio (cacheada y bien posicionada), la tarea el final, y el volumen de baja señal el medio, donde menos daño hace.

La regla de cierre

Nunca escribas nada después de la tarea. Ni un recordatorio, ni un «responde en formato X», ni una repetición de las reglas. Cualquier coletilla la empuja al medio. Si una restricción es crítica y hay que repetirla, va arriba —donde ya está cacheada y bien posicionada—, no como cierre.

Un matiz que ahorra dinero: cachear solo el contexto estable rinde más que cachear todo el contexto, que paradójicamente puede aumentar la latencia. Y hay una ventana de reutilización: partir una tarea en varios días paga el prefijo cada vez.

10 · Contra la invención

Tres técnicas, de la más barata a la más cara. Elige por lo que cuesta el error, no por costumbre.

10.1 · Anclaje a fuentes

Dos partes, y la segunda es la que casi todo el mundo olvida:

Responde ÚNICAMENTE con base en .
Si la información no se encuentra en él, responde que no está disponible.

Nombra el documento, no digas «los adjuntos»: con varias fuentes se enfoca en una y te deja fuera el resto sin avisar.

La segunda cláusula es la decisiva: darle una salida honesta. Un modelo al que le falta información y no tiene forma legítima de decirlo rellena el hueco con algo plausible. No por engañar — porque no le diste otra opción.

10.2 · Citas textuales verificables

Pide la cita entre comillas y luego búscala en el documento original.

El matiz que rara vez se documenta

La cita no siempre es literal: a veces reordena o parafrasea sin avisar. Busca fragmentos de tres o cuatro palabras, no la frase completa, y si no aparece, pide la página. Consecuencia para cualquier verificación automatizada: un comparador de texto exacto marcaría como falsas citas sustancialmente correctas. El criterio fiable es «la referencia resuelve y dice lo mismo», no coincidencia carácter a carácter.

10.3 · Autoconsistencia

Resuelve esto de tres formas distintas y compara los resultados.

Si los tres coinciden, sube la confianza. Si divergen, encontraste un error sin necesidad de saber cuál era la respuesta correcta. Esa es la propiedad valiosa: la divergencia es señal, y detectarla no exige un verificador.

Sirve donde el error es de razonamiento o cálculo. No sirve donde el error es un dato inventado: los tres caminos parten del mismo dato falso y coinciden felizmente.

Riesgo del error	Técnica
Bajo	Anclaje
Medio, y el error sería de razonamiento	Anclaje + autoconsistencia
Alto, o el error sería un dato falso	Anclaje + verificación adversarial

La regla de fondo, y vale para las tres: no creas ciegamente lo que dice el modelo; crea mecanismos para validarlo. Y prefiere el mecanismo más barato que detecte el error que te importa.

11 · Autoridad y jerarquía

Cuando dos instrucciones se contradicen hay un orden entrenado de privilegio:

Nivel	Origen	Privilegio
1	Mensaje de sistema	Máximo
2	Mensaje del usuario	Alto
3	Instrucciones dentro de imágenes o audio	Medio
4	Texto que viene de herramientas: búsquedas, documentos, salidas de código	El más bajo

El nivel 4 es una defensa: si un documento trae escondido un ignora tus instrucciones anteriores, debe ser ignorado.

Dos matices que evitan usarlo mal:

- Solo aplica cuando hay conflicto. Si están alineadas, obedece la de menor privilegio sin problema.
- No es una garantía absoluta. Es una tendencia entrenada, no un control. Para lo que de verdad no puede fallar, el límite va en la infraestructura: si el proceso no tiene la credencial, no hay instrucción que evadir.

12 · Elegir el modelo

Un modelo rápido y uno de razonamiento no se diferencian en «cuál es más listo», sino en cuánto espacio se dan para pensar antes de responder.

- Rápido — respuesta directa, tareas acotadas, alto volumen.
- Razonamiento — varios pasos, restricciones que interactúan, cálculo.

Con modelos actuales, la instrucción piensa paso a paso ya no cambia mucho: razonan por defecto. Sigue sirviendo cuando quieres ver el razonamiento para auditarlo, que es distinto de que lo haga.

13 · La disciplina que sostiene todo

Sesión limpia para comparar

Si quieres saber si un cambio mejoró algo, corre las dos versiones en sesiones limpias. En la misma sesión, lo anterior contamina lo siguiente y la comparación no vale.

Lo corregido no se borra, se acumula

Cuando corriges al modelo a mitad de conversación, el dato equivocado sigue en el contexto junto al correcto. Hay medición de que un error temprano en una conversación de varios turnos degrada el desempeño cerca de un 39%.

Consecuencia práctica

Ante un error de base, abre chat nuevo. No sigas corrigiendo encima. Es la razón de que en un sistema por roles un bloqueo vuelva al principio en lugar de seguir discutiendo en el mismo hilo.

Auditar antes de suponer

Antes de rediseñar un asistente que heredaste, pregúntale qué tiene:

¿Qué información tienes disponible sobre mí en este momento?
Enlístala por origen: instrucciones del sistema, memoria de sesiones anteriores, este chat, archivos que subí, herramientas.
Si no tienes acceso a alguna de esas fuentes, dilo explícitamente.

Lo interesante no es lo que dice que tiene: es lo que dice que no tiene. Ahí están los huecos que nadie te avisó.

Cuando encuentres uno, escríbelo con esta forma — obliga a nombrar el supuesto, que es la parte que normalmente queda tácita:

«Yo asumía que sabía ____, y resulta que no está en ningún lado del contexto.»

14 · Los nueve errores más comunes

#	Error	Qué hacer
1	Culpar al modelo por una delegación mala	Si a una persona nueva le hubiera faltado ese dato, al modelo también
2	Confundir problema de contexto con problema de redacción	Antes de reformular otra vez, pregunta qué información falta
3	Pedir «corto», «profesional», «bien»	Adjetivos sin unidad. Cuantifica
4	Acumular instrucciones	Hay recortes documentados de más del 80% de un prompt de sistema que mejoraron el resultado
5	Corregir encima cuando el error es de base	Chat nuevo
6	Crear la cita sin buscarla — o descartarla porque no coincide letra a letra	Busca fragmentos cortos; pide la página
7	No darle salida honesta	Sin un «di que no está disponible», inventa
8	Comparar en la misma sesión	Contamina y no te enteras
9	Agrandar el prompt en vez de encadenar	Divide el problema en estaciones, cada una un chat nuevo

15 · Dónde seguir

Documento	Cuándo leerlo
FOUNDATIONS.md	Este documento, en el catálogo de skills
AUTHORING.md	Al escribir una skill nueva: anatomía y formato
CONTEXT.md	Al decidir qué información entra a la ventana
PRECISION.md	Al diseñar una skill cuya salida deba ser comprobable
Guía del Framework (PDF)	Al implantar el ciclo de vida por roles en un equipo

Procedencia de las cifras

Los datos citados —46 frente a 19 herramientas, caída del 39% por error temprano, 70-75% frente a 55-60% según posición, recorte de más del 80% de un prompt de sistema— provienen de material formativo y no han sido verificados de forma independiente. Trátales como órdenes de magnitud que orientan una decisión, no como constantes. Las prácticas se sostienen aunque las cifras exactas difieran.